

FORMATION PYTHON

Premier programme

Bonjour tout le monde

```
>>> print("Bonjour tout le monde")
Bonjour tout le monde
>>>
```

Nous pouvons mettre cette ligne dans un fichier

```
# bonjour.py
print("Bonjour tout le monde")
```

Lancer ce fichier

```
fasm% python3 Bonjour.py
```

Nous pouvons utiliser le #!

```
#!/usr/bin/env python3
print("Bonjour tout le monde »)"
```

Environnement de Développement

- Eric
- SPE
- Komodo
- Pydev + eclypse
- PyCharm

Variables et Expressions

Expressions

```
>>> 3+ 5
>>>3 + (5*4)
>>>3 ** 2
>>>'Bonjour' + ' le monde'
```

Assignation des Variable

```
a = 4
b = a * 4.5
c = (a+b)/2.5
a = "Bonjour tout le monde"
```

Les variables sont typées dynamiquement

Les Variables sont justes des noms pour des object.

Types basiques

Nombres

```
a = 3              #entier
b = 4.5            # flottant
c = 517288833333L  # entier long
d = 4 + 3j # nombres complexes
```

Chaînes de caractères

```
a = ’Bonjour’
b = "le monde"
c = "robert dit ’Salut les gars.’"  # mélange de ' et "
d = ’’’Les triples quotes
Permettent d’écrire
Sur plusieurs lignes’’’
e = """ça marche aussi pour les doubles quotes"""
```

type()
Len()
str()
ord()
hex()
bin()
int()
chr()

Indentation

- Pas de délimiteur de bloc => indentation

```
Bloc
    Instruction
    Instruction
    Instruction
Autre Bloc
    Instruction
    Bloc dedans
        Instruction
        Instruction
    instruction
```

Conditions

if-else

```
if a < b:
      z= b
else:
      z= a
```

==	égal
!=	différent
<	inférieur
>	supérieur
<=	inférieur ou égal
>=	supérieur ou égal

pass

```
if a < b:
   pass       # ne rien faire
else:
     z= a
```

Conditions

elif

```
if a == ’+’:
    op = 'PLUS'
elif a == ’-’:
    op = 'MOINS'
elif a == ’*’:
    op = 'MULTIPLICATION'
else:
     op ='INCONNU'
```

Il n'existe pas switch comme dans certains langages

Expressions Booleénnes: and, or, not

```
if b >= a and b <= c:
    print "b est entre  a et c"
if not (b < a or b > c):
    print "b est toujours entre  a et  c"
```

a	b	a and b	a or b
False	False	False	False
False	True	False	True
True	False	False	True
True	True	True	True

Listes

Différentes listes

a = [2, 3, 4] # liste d'entiers

b = [2, 7, 3.5, "Hello"] # liste mixte

c = [] # liste vide

d = [2, [a,b]] # liste contenant une liste

e = a + b # concaténer deux listes

append()
pop()
Insert()
sort()
remove()
reverse()

List Manipulation

x = a[1] # récupère le 2 éme élément de la liste dans x

y = b[1:3] # retourne une partie de la liste (1 inclus, 3 exclus)

z = d[1][0][2] # listes emboitées

B[0] = 42 # remplace l'élément 0 de la liste par 42

Tuples

Tuples

f= (2,3,4,5) # un tuple d'entiers

g= () # un tuple vide

h= (2, [3,4], (10,11,12)) # un tuple contenant un mixte d'objets

Manipulation de Tuple

x = f[1] # place le contenu de l'élément 2 de la liste f dans x (x=3)

y = f[1:3] # y = (3,4)

z = h[1][1] #. z = 4

Commentaire

Les tuples sont comme les listes, mais leur taille est fixée lors de leur création. Elles sont immutables.

Dictionnaires

Dictionnaires

```
A= { } # dictionnaire vide

b = { 'x': 3, 'y': 4 }

c = { 'uid': 105,
        'login': 'franck',
          'name' : 'ebel'
           }
```

Dictionnaires

Accès aux dictionnaires

```
u = c[’uid’] # récupére un élément
c[’shell’] = "/bin/sh" # place un élément
for var in d :
        If var=='a' :
                print( "'a' existe dans le dictionnaire ')
d = c.get("directory",None) # la même chose en plus compacte
```

set

Un set est un ensemble, c'est-à-dire une collection non ordonnée d'éléments uniques, ce qui se révèle très pratique dans beaucoup d'usages courants. Ce nouveau type conteneur étant fourni en mutable (comme les listes), et immutable (comme les tuples).

```
>>> s_mutable = set([1,2,3])

>>> s_immutable = frozenset([1,2,3])
```

Nous pouvons utiliser les accolades , comme les dictionnaires { et } pour une énumération sans risque de confusion :

```
>>> s = {1, 2, 3}

>>> s

{1, 2, 3}
```

set

Il reste cependant une petite exception qui rend cette solution syntaxique imparfaite, et potentiellement génératrice d'erreurs ou d'incompréhensions, c'est le cas de l'ensemble vide. En effet, {} ne peut pas représenter à la fois le dictionnaire vide et le set vide :

```
>>> s = {}

>>> s

{}

>>> isinstance(s, set)

False

>>> isinstance(s, dict)

True
```

set

De manière logique et rétrocompatible, {} représente donc toujours le dictionnaire vide, et c'est set() qui permet de créer un set vide :

```
>>> s = set()

>>> s

set()

>>> s.add(1)

>>> s

{1}
```

Les boucles

La boucle while

```
while a < b:
        a=a+ 1
```

La boucle for

```
for i in [3, 4, 10, 25]:
        print(i)
for c in "Bonjour tout le monde":
         print(c)
for i in range(0,100):
          print(i)
```

Exceptions

La commande try

```
try:
    f = open("fichier.txt")
except IOError:
    print("impossible d'ouvrir fichier.txt’.")
```

La commande raise

```
def factoriel(n):
        if n < 0:
                raise ValueError,"Exception valeur negative"
        if (n <= 1): return 1
                else: return n*factoriel(n-1)
```

Exceptions

exception

```
>>> factoriel(-1)
Traceback (innermost last):
File "<stdin>", line 1, in ?
  File "<stdin>", line 3, in factoriel
ValueError: Expected non-negative number
>>>
```

Le mot clé with

L'instruction `with` permet de beaucoup simplifier ce genre de codes en les transformant en ceci (qui réalise strictement la même chose)

```
f= open('fichier', 'w')
try:
    # Opérations sur f
except Exception:
    # Une erreur a eu lieu, on la gère (ou non)
finally:
    # On referme notre fichier
f.close()
```

La plupart du temps pour gérer les erreurs, nous utilisons l'exemple ci dessus.

```
with open('fichier', 'w') as f:
        #operation sur f
```

Ce code réalise vraiment la même chose que les blocs `try` au-dessus, exceptée la possibilité de gérer nous-mêmes les erreurs. Cela passe par l'utilisation des contexts managers, des objets possédant deux méthodes magiques : `__enter__` et `__exit__` appelées à l'entrée ou à la sortie d'un bloc `with`.

Fichiers

La fonction open()

```
f = open("fichier","w")       # Ouvre le fichier en ecriture
g = open("fichier2","r")       # Ouvre le fichier en lecture
```

Lire et écrire des données

```
f.write("Bonjour le monde")
data = g.read() # lit toutes les données
line = g.readline() # lit une seule ligne
lines = g.readlines()  # lit les données comme une liste de lignes
```

Module

Operating system

http://docs.python.org/2/library/os.html

os.chdir(path) : change de dossier , path est le chemin voulu

os.getcwd() : récupère le chemin courant

os.ctermid() : donne le terminal courant

os.getgid() : donne le l'identifiant du groupe

os.getuid() : donne l'identifiant de l'utilisateur

os.system() : exécute la commande système

os.getsize() : récupère la taille d'un élément

Module

```
import os
>>> os.path.join("/home/febel/", "python.odt")
'/home/febel/python.odt'
>>> os.path.expanduser("~")
'/home/febel'
>>> os.path.join(os.path.expanduser("~"), "Python")
'/home/febel/Python'
>>> os.path.split("/home/febel/python.odt")
('/home/febel', 'python.odt')
```

```
>>> (filepath, filename) = os.path.split("/home/febel/python.odt")
>>> filepath
'/home/febel'
>>> filename
'python.odt'
>>> (shortname, extension) = os.path.splitext(filename)
>>> shortname
'python'
>>> extension
'.odt'
```

Module

```
>>> os.listdir("/home/febel/Documents")
>>> dirname = "/home/"
>>> os.listdir(dirname)
>>> [f for f in os.listdir(dirname)
...      if os.path.isfile(os.path.join(dirname, f))]
>>>
```

for root, dirs, files in os.walk(path)

Exercices

1 . Le langage geek a cela de particulier que chaque lettre de l'alphabet est remplacé par un autre symbole que nous pouvons récupérer sur le web avec une recherche Google par exemple.

Il existe plusieurs alphabets geek, nous prendrons celui du tableau 1 trouvé sur wikipedia, un peu simplifié.

Tableau 1

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
4	8	(	d	3	f	6	#	1	j	k	1	m	n	0	p	q	2	5	7	u	v	w	x	y	z

Nous allons donc devoir créer un programme qui remplace pour chaque lettre de l'alphabet traditionnel, la lettre geek.

2.Nous voudrions créer un script qui demande à l'utilisateur une adresse IP.

Cette adresse Ip sera utilisée comme base pour déterminer si sur la plage de cette machines (en masque 255.255.255.0), les ordinateurs sont allumés ou non.

Pour cela nous utiliserons la commande ping que nous lancerons grâce au module os de Python.

Le module string

Quelques fonctions du module string

- string.atof(s) : conversion en flottant
- string.atoi(s) : conversion en entier
- string.atol(s) : conversion en entier long
- string.count(s,pattern) ; compte les occurrences de pattern dans s
- string.find(s,pattern) : trouve un pattern dans s
- string.split(s, sep) : éclate une chaîne de caractère en fonction du séparateur
- string.join(strlist, sep) : reconstruit une chaîne de caractère en fonction du séparateur
- string.replace(s,old,new) : remplace les occurrences anciennes par des nouvelles dans s
- String.upper() : tout en minuscule
- String.lower() : tout en majuscule

Module string

Exemples

```
>>> var="Bonjour le Monde"
>>> var.split()
['Bonjour', 'le', 'Monde']
>>> var.replace("Bonjour", "au revoir")
'au revoir le Monde'
>>> var1
['Bonjour', 'le', 'Monde']
>>> " ".join(var1)
'Bonjour le Monde'
>>>
```

Module sys

sys.path

Liste des dossiers dans lesquels Python cherche les modules à importer .

sys.modules

Liste des modules importés par le programme .

sys.version

Donne des informations sur la version de Python qui est utilisée.

sys.version_info

Donne, sous forme d'un tuple, des informations sur la version de Python. Cette forme peut être plus pratique à utilisée lorsqu'il s'agit de tester si la version utilisée correspond à certains critères.

```
>>> if sys.version_info[0] < 3:
...     print('La version de Python que vous utilisez est trop ancienne.')
```

Module sys

sys.float_info

Donne, sous forme d'un objet, des informations sur les nombres à virgule du système (précision, taille maximale…).

sys.getfilesystemencoding()

Renvoie le nom de l'encodage utilisé par le système.

Sys.argv

Recupére les arguments

Sys.plateform

Donne la plateforme (systeme d'exploitation)

>>> sys.version

'3.3.2+ (default, Sep 19 2013, 14:57:27) \n[GCC 4.8.1]'

>>> sys.version_info

sys.version_info(major=3, minor=3, micro=2, releaselevel='final', serial=0)

Module re

Expression régulière = reconnaissance d'expression

Caractères spéciaux:

- Méta-caractères
- Facteurs d'occurence

Les méta-caractères:

. Caractère quelconque sauf fin de ligne.

$ Fin de ligne.

^ Début de ligne, sauf dans [], négation

[] Un des caractères contenus dans les crochets.

- Intervalle de caractères dans []

Module re

- Les facteurs d'occurences:

? Expression régulière précédente optionnelle

* Répétition 0, 1 ou plusieurs fois

+ Répétition 1 ou plusieurs fois

{n} Répétition de exactement n fois

{n,} Répétition de au moins n fois

{n,m} Répétition de au moins n mais au plus m fois

| Alternative

\c Suppression de la spécificité du caractère 'c'

() Groupement d'expressions

Module re

ab(cd)*ef abcdef, abcdcdef, abcdefg, abef

ab[c|d]ef abcef, abdef

ab[^a--z]ef ab1ef, abCef

chat|chien|poisson = sort ‘chat’, ‘chien’ ou ‘poisson’ (va sortir

‘chaton’ ou ‘achat’)

\b(chat|chien)\b = sort ‘chat’ ou ‘chien’ en tant que mots complets seulement

(anti)(gel)? = va retourner ‘anti’, ‘antigel’, mais pas ‘gel’

car?pe = va trouver ‘carpe’ et ‘cape’

\b[1--9][0--9]{3}\b = donne un nombre entre 1000 et 9999

\b[1--9][0--9]{2,4}\b = donne un nombre entre 100 et 99999

Module re

```
>>>import re
>>> chaine = "La formation python "
>>> re.search('python',chaine)
<_sre.SRE_Match object at 0x40365218>
>>> if re.search('python',chaine):
...    print "L'expression reguliere fonctionne !!!"
```

Module re

```
>>> chaine = "La formation python"
>>> re.search('python',chaine)
<_sre.SRE_Match object at 0x7ff444f0>
>>> re.match('python',chaine)
>>>
```

Module re

Il est aussi commode de préalablement compiler l'expression régulière à l'aide de la fonction compile qui renvoie un objet de type expression régulière :

```
>>> maregex = re.compile('^python')

>>> maregex

<_sre.SRE_Pattern object at 0x7ff32cf0>
```

Module re

On peut alors utiliser directement cet objet avec la méhode search() :

```
>>> chaine="python est le langage que nous apprenons"
>>> maregex.search(chaine)
<_sre.SRE_Match object at 0x7ff44410>
>>> chaine="La formation python"
>>> maregex.search(chaine)
```

Module re

Vous vous posez la question, pourquoi python renvoie une instance MatchObject lorsqu'une expression régulière matche une chaîne ?

Et bien parce qu'on peut réutiliser cet objet pour récupérer des informations sur les zones qui ont été matchées :

Module re

```
>>> regex = re.compile('([0-9]+)\.([0-9]+)')
>>> MATCH = regex.search("pi : 3.1415926535897931")
>>> MATCH.group(0)
'3.1415926535897931'
>>> MATCH.group(1)
'3'
>>> MATCH.group(2)
'1415926535897931'
>>> MATCH.start()
5
>>> MATCH.end()
23
```

Module re

On s'aperçoit sur cet exemple que l'utilisation de la fonction group() sur une instance MatchObject permet de récupérer les zones situées entre parenthèses. Les fonctions start() et end() donne respectivement la position de début et de fin de la zone qui matche la regex.

Module re

Il est important de noter que la fonction search() ne renvoie que la première zone qui matche, même s'il en existe plusieurs :

```
>>> MATCH = regex.search("pi : 3.1415926535897931 , et e :
2.7182818284590451")
>>> MATCH.group(0)
'3.1415926535897931'
```

Module re

Si on veut récupérer chaque zone, on peut utiliser la fonction findall qui renvoie un tuple avec l'ensemble de ces zones :

```
>>> MATCH = regex.findall("pi : 3.1415926535897931 , et e :

2.7182818284590451")

>>> MATCH

[('3', '1415926535897931'), ('2', '7182818284590451')]

>>>
```

Module re

Enfin, la fonction sub(repl,string) permet d'effectuer des remplacements assez puissants. Par défaut la fonction sub remplace toutes les occurrences de la regex; si on souhaite que les n premières occurrents, on peut utiliser l'argument count=n :

>>> regex.sub('3.14',"pi : 3.1415926535897931 , et e :

2.7182818284590451")

'pi : 3.14 , et e : 3.14'

>>> regex.sub('3.14',"pi : 3.1415926535897931 , et e :

2.7182818284590451",count=1)

'pi : 3.14 , et e : 2.7182818284590451'

Exercices

Exercice 1

Écrire un programme python filter.py qui fonctionne en ligne de commande et qui :

– prend deux arguments

– le premier est une expression régulière

– le deuxième est le nom d'un fichier texte

– affiche les lignes du fichier texte qui correspondent à l'expression régulière

Exemple soit le fichier test.txt suivant :

bonjour

12, le nombre est 12

au revoir

27

On aura alors par exemple
> python filter.py re test.txt
12, le chiffre est 12
27
>

Exercices

Exercice 2

Nous allons créer un fichier texte contenant des adresse mails .

Nous voudrions créer un script qui recherche ces mails et nous les inscrit dans un fichier texte.

Exercice 3 et 4

- Exercice 3

Nous voudrions créer un script qui effectue un ifconfig (ou ipconfig) et qui nous donne l'adresse mac des cartes réseaux.

- Exercice 4

Nous voudrions écrire un script qui demande à l'utilisateur un chemin puis une regex.

Le script recherchera dans le repertoire et les sous répertoires les correspondances à la regex et nous les affichera à l'écran.

module

Définition

Les modules sont des programmes Python qui contiennent des fonctions que l'on est amené à réutiliser souvent (on les appelle aussi bibliothèques ou librairies).

Importation de modules

```
>>> import random
>>> random.randint(0,10)
4
```

module

À l'aide du mot-clé from, vous pouvez importer une fonction spécifique d'un module donné. Remarquez que dans ce cas il est inutile de répéter le nom du module, seul le nom de ladite fonction est requis.

```
>>> from random import randint
```

Nous pouvons également importer toutes les fonctions d'un module :

```
>>> from random import *

>>> randint(0,50)

46
```

module

Dans la pratique, plutôt que de charger toutes les fonctions d'un module en une seule fois (from random import *), il est conseillé de charger le module (import random) puis d'appeler explicitement les fonctions voulues (random.randint(0,2)).

il est également possible de définir un alias pour un module :

```
>>> import random as rand

>>> rand.randint(1, 10)

6
```

module

Obtenir de l'aide sur les modules importés

Pour obtenir de l'aide sur un module rien de plus simple, il suffit d'invoquer la commande

help() :

```
>>> import random
>>> help(random)
...
```

module

La commande help() est en fait une commande plus générale permettant d'avoir de l'aide sur n'importe quel objet chargé en mémoire.

```
>>> x = range(2)

>>> help(x)

Help on list object:

class list(object)

| list() -> new list

| list(sequence) -> new list initialized from sequence's items
```

module

si on veut connaître en seul coup d'oeil toutes les méthodes ou variables associées à un objet, on peut utiliser la commande dir :

```
>>> import random
>>> dir(random)
```

Fonction

Pour définir une fonction, Python utilise le mot-clé def et si on veut que celle-ci renvoie une valeur, il faut utiliser le mot-clé return. Par exemple :

```
>>> def carre(x):
...         return x**2
...
>>> print(carre(2))
4
>>> res = carre(2)
>>> print(res)
4
```

fonction

une fonction ne prend pas forcément un argument et ne renvoie pas forcément une valeur, par exemple :

```
>>> def hello():
... print "bonjour"
...
>>> hello()
bonjour
```

fonction

Passage d'arguments

```
>>> def fois(x,y):
...         return x*y
...
>>> fois(2,3)
6
>>> fois(3.1415,5.23)
16.430045000000003
>>> fois('to',2)
'toto'
```

fonction

Un autre gros avantage de Python est que ses fonctions sont capables de renvoyer plusieurs valeurs à la fois, comme dans cette fraction de code :

```
>>> def carre_cube(x):
…             return x**2,x**3
…
>>> carre_cube(2)
(4, 8)
```

fonction

```
>>> def carre_cube2(x):
... return [x**2,x**3]
...
>>> carre_cube2(3)
[9, 27]
```

Il est possible de passer un ou plusieurs argument(s) de manière facultative et de leur attribuer une valeur par défaut :

```
>>> def autre_fct(x=1):
…             return x
...
>>> autre_fct()
1
>>> autre_fct(10)
10
```

fonction

Portée des variables

on peut créer des variables au sein d'une fonction qui ne seront pas visibles à

l'extérieur de celle-ci ; on les appelle variables locales. Observez le code suivant :

```
>>> def mafonction():
...             x = 2
...             print('x vaut',x,'dans la fonction')
...
```

fonction

```
>>> def mafonction(x):
...             print('x vaut',x,'dans la fonction')
...
>>> mafonction(2)
x vaut 2 dans la fonction
>>> print x
Traceback (most recent call last):
File "<stdin>", line 1, in ?
NameError: name 'x' is not defined
```

fonction

Lorsqu'une variable déclarée à la racine du module , elle est visible dans tout le module.

```
>>> def mafonction():
...             print(x)
...
>>> x = 3
>>> mafonction()
3
>>> print x
3
```

fonction

Si nous voulons vraiment modifier une variable globale dans une fonction, il faut utiliser le mot-clé global :

```
>>> def mafonction():
...         global x
...         x = x + 1
...
>>> x=1
>>> mafonction()
>>> x
2
```

Exercice

Ecrire le script du jeu de devinette suivant :

>>>

Le jeu consiste à deviner un nombre entre 1 et 100 :

---> 50

trop petit !

---> 75

trop petit !

---> 87

trop grand !

---> 81

trop petit !

---> 84

trop petit !

---> 85

Gagné en 6 coups !

```
variable=Générer un nombre aléatoire
Compteur=0
solutiontrouvee=False
Tant que False
        Demander un nbr a l'utilisateur
        Si le nombre plus petit :
                Indiquer plus petit
                Incrementer compteur
        Si le nombre est plus grand :
                Indiquer plus grand
                Incrementer compteur
        Si le nombre est égal :
                solutiontrouvee=True
Indiquer le nombre de coups joués
```

Fonction filter

La fonction filter applique la fonction passée en premier argument sur chacun des éléments de la séquence passée en second argument et retourne une nouvelle liste qui contient tous les éléments de la séquence pour lesquels la fonction a retourné une valeur vrai.

```
>>> def odd(n):                1
...         return n % 2
...
>>> li = [1, 2, 3, 5, 9, 10, 256, -3]
>>> list(filter(odd, li))          2
[1, 3, 5, 9, -3]
```

Fonction map

```
>>>def double(n):
  …          return n*2
li = [1, 2, 3, 5, 9, 10, 256, −3]
>>> map(double, li)
[2,4, 6, 10, 18, 20, 512, −6]
>>> [double(n) for n in li]
[2,4, 6, 10, 18, 20, 512, −6]
>>> newlist = []
>>> for n in li:
…        newlist.append(double(n))
>>> newlist
[2,4, 6, 10, 18, 20, 512, −6]
```

Fonction lambda

lambda peuvent être employées partout où une fonction est nécéssaire.

```
>>> def f(x):
...         return x*2
...
>>> f(3)
6


>>> g = lambda x: x*2
>>> g(3)
6
>>> (lambda x: x*2)
6
```

exercices

Exercice

Écrire une fonction utilisant lambda qui calcul a partir d'une liste contenant des températures en degré l'équivalence en farenheit.

Création de module

Vous pouvez créez votre propres modules très simplement en Python. Il vous suffit d'écrire un ensemble de fonctions (et/ou de variables) dans un fichier, puis d'enregistrer celui-ci avec une extension .py

Création de module

```
"""Module qui affiche des messages """
def Bonjour(nom):
    """Affiche bonjour ! """
    return "bonjour " + nom
def Ciao(nom):
    """Affiche ciao ! """
    return "ciao " + nom
def Hello(nom):
    """Affiche hello ! """
    return "hello " + nom + " !"
date=16092008
```

```
>>> import message
>>> message.Hello("Joe")
'hello Joe !'
>>> message.Ciao("Bill")
'ciao Bill'
>>> message.Bonjour("Monsieur")
'bonjour Monsieur'
>>> message.date
16092008
```

Création de module

Les commentaires (entre triple guillemets) situés en début de module et sous chaque fonction permettent de fournir de l'aide invoquée ensuite par la commande help() :

Help on module message:

NAME

message - Module qui affiche des messages

FUNCTIONS

Bonjour(nom)

Affiche bonjour !

Ciao(nom)

Affiche ciao !

Hello(nom)

Affiche hello !

DATA

date = 16092008

FILE

/home/febel/Documents/linagora/message.py

Classes

Une classe permet de définir des objets qui sont des représentants (des instances) de cette classe. Les objets peuvent posséder des attributs (variables associées aux objets) et des méthodes (fonctions associées aux objets).

Exemple de la classe Rectangle :

```
class Rectangle:

    "ceci est la classe Rectangle"

    # initialisation d'un objet

    # définition des attributs avec des valeurs par défaut

    def __init__(self, long = 0.0, larg = 0.0, coul = "blanc"):

            self.longueur = long

            self.largeur = larg

            self.couleur = coul

    # definition de la méthode qui calcule la surface
```

```
def calculSurface(self):
                        print("surface = %.2f m2" %(self.longueur * self.largeur))
    # definition de la méthode qui transforme un rectangle en carré
    def changeCarre(self, cote):
                         self.longueur = cote
                         self.largeur = cote
```

Classes

Tous les attributs et toutes les méthodes se réfèrent toujours à self qui désigne l'objet lui même. Attention, les méthodes prennent toujours au moins self comme argument.

Exemples d'utilisation de la classe Rectangle :

```
# création d'un objet Rectangle avec les paramètres par défaut

rect1 = Rectangle()

print(rect1.longueur, rect1.largeur, rect1.couleur)

# sans surprise :

rect1.calculSurface()

# on change le rectangle en carré

rect1.changeCarre(30)

rect1.calculSurface()

# création d'un objet Rectangle avec des paramètres imposés

rect2 = Rectangle(2, 3, "rouge")

rect2.calculSurface()
```

Classes

La classe

```
class compte:
        def __init__(self, initial):
                self.balance = initial
        def depot(self, amt):
                 self.balance = self.balance + amt
        def retrait(self,amt):
                self.balance = self.balance - amt
        def solde(self):
                return self.balance
```

Comment utiliser cette classe ?:

```
a = compte(1000.00)
a.depot(550.23)
a.depot(100)
a.retrait(50)
print(a.solde())
```

Héritage et polymorphisme

une classe Python peut toujours être spécialisée en une classe fille qui hérite alors de tous les attributs (données et méthodes) de sa super classe. Comme tous les attributs peuvent être redéfinis, une méthode de la classe fille et de la classe mère peut posséder le même nom mais eă ectuer des traitements diă érents (surcharge) et l'objet s'adaptera dynamiquement, dès l'instanciation

Héritage et polymorphisme

```
>>> class Quadrupede:
...         def piedsAuContactDuSol(self):
...                 return 4
...
>>> class QuadrupedeDebout(Quadrupede):
...                 def piedsAuContactDuSol(self):
...                         return 2
...
>>> q1 = Quadrupede()
>>> q1.piedsAuContactDuSol()
4
>>> q2 = QuadrupedeDebout()
>>> q2.piedsAuContactDuSol()
2
```

Exercice

Exercice

Définissez une classe Cercle(). Les objets construits à partir de cette classe seront des cercles de tailles variées. En plus de la méthode constructeur (qui utilisera donc un paramètre rayon), vous définirez une méthode surface(), qui devra renvoyer la surface du cercle.

Définissez ensuite une classe Cylindre() dérivée de la précédente. Le constructeur de cette nouvelle classe comportera les deux paramètres rayon et hauteur. Vous y ajouterez une méthode volume() qui devra renvoyer le volume du cylindre (rappel : volume d'un cylindre = surface de section × hauteur).

Exemple d'utilisation de cette classe :

```
>>> cyl = Cylindre(5, 7)

>>> print(cyl.surface())

78.54

>>> print(cyl.volume())

549.78
```